Introduction

Data cleaning is a fundamental but often under-acknowledged stage in many civic, academic, and administrative data workflows. Public datasets released by governments, NGOs, and research organisations frequently contain inconsistencies in place names, such as spelling variations, abbreviations, and legacy naming conventions. These inconsistencies make it difficult to merge datasets, identify duplicates, or perform reliable analysis. While large-scale geocoding and mapping services such as Google Geocoding API, ESRI ArcGIS, or Mapbox exist, they typically require structured geographic inputs, API keys, internet connectivity, or specialised geospatial expertise. In practice, many data practitioners work with simpler tabular datasets that contain only textual place names and limited contextual metadata, often within spreadsheet-based tools.

Numerous civic technology initiatives and non-profit data projects have reported difficulty when reconciling place names that are colloquial, multilingual, or inconsistently recorded. A 2022 report by MIT's Civic Data Design Lab **[9]** describes how NGOs working with open mapping tools face persistent challenges integrating datasets with locally variant place names, especially in underserved or non-standardised regions. At the same time, geocoding platforms such as Google Maps and Mapbox have been shown to exhibit systematic coverage gaps and name resolution biases, particularly in rural areas or the Global South. The Oxford Internet Institute (2021) **[10]** highlights how these services may underperform or misrepresent places for marginalised populations due to language mismatches or prioritised commercial datasets. These limitations highlight the need for tools like CleanLink+ that offer more flexible, transparent, and locally relevant reconciliation capabilities.

OpenRefine is widely used in such contexts as a lightweight, interactive tool for cleaning and reconciling tabular data. One of its key features is the ability to reconcile column values against external reference datasets via a reconciliation API. This project is based on the "Reconciliation Service for a Dataset" project template, and aims to design, implement, and evaluate a custom reconciliation service that supports the cleaning of messy place-name data within OpenRefine. Rather than attempting to replace full geocoding services, the goal is to support an earlier stage of data preparation: resolving ambiguous or inconsistent place names against a trusted reference dataset using approximate string matching and contextual constraints.

The project is titled CleanLink+ and it implements a reconciliation service for city and settlement names. The service exposes a locally hosted API compatible with OpenRefine's reconciliation protocol and allows users to match messy place names to a reference dataset derived from public geographic data. The system is designed to support approximate matching, alternate name resolution, and multi-field matching using additional contextual attributes such as country codes and regional identifiers. This design choice directly addresses common ambiguity cases such as multiple places sharing the same name across different countries or administrative regions.

A key motivation for this project is the observation that simple string matching is often insufficient when dealing with real-world geographic data. For example, a place name such as “Springfield” or “Vila” may correspond to dozens of distinct locations globally. Similarly, alternate names, diacritics, transliterations, and informal spellings (e.g. “Surudeu” for “Soldeu”) frequently appear in civic datasets. CleanLink+ therefore goes beyond single-field matching by incorporating alternate names from the reference dataset and by allowing additional fields to be supplied during reconciliation. These contextual fields are optional, enabling the service to remain flexible while still improving disambiguation when such information is available.

From a technical perspective, the project intentionally balances simplicity with scalability. Early versions of the prototype used an in-memory CSV file as the reference dataset, which was sufficient for demonstrating basic functionality but did not scale well. The current implementation uses a SQLite-backed database, allowing the system to handle larger datasets more efficiently and directly addressing the previous disadvantages. The matching logic is implemented using the RapidFuzz **[6]** library which provides efficient approximate string matching algorithms suitable for this use case.

An important aspect of the project is evaluation. Rather than relying on arbitrary thresholds, the system’s matching performance is assessed using a curated set of test queries with known expected matches. Precision, recall, and F1-score are computed to analyse the impact of different confidence thresholds and matching strategies. This evaluation process directly informed design decisions such as selecting an appropriate match confidence threshold and balancing false positives against missed matches. By explicitly analysing these trade-offs, the project moves beyond the prototype phase and starts with the practical challenges of reconciliation quality.

In summary, CleanLink+ aims to provide a practical, extensible reconciliation service that supports the cleaning of messy place-name data within OpenRefine. The project focuses on lightweight deployment, transparent matching logic, and proper evaluation, making it suitable for civic and administrative data preparation workflows where full geocoding solutions may be overkill. The remainder of this report details the design decisions, implementation, evaluation methodology, and limitations of the system, along with ideas for future improvement.

Literature Review

One of the most important parts of this project is fuzzy string matching, it’s the core of how CleanLink+ is designed to work. Since we’re trying to match messy location names (like city or district names), I spent time looking into which algorithms, libraries, and real-world tools are best suited for this kind of task.

The main library I’ve chosen is RapidFuzz. It’s a fast, memory-efficient string matching library for Python (and C++), and it’s basically an improved version of older libraries like FuzzyWuzzy. RapidFuzz implements standard similarity algorithms like Levenshtein, Damerau, and token-based matching, but in a much faster way. For my needs, matching things like “San Fransisco” to “San Francisco”, this library is more than enough. Elmobark’s 2025 benchmark [2]

compared a variety of fuzzy match libraries and showed RapidFuzz had excellent accuracy (~96–97%) while being 40% faster than alternatives and far more memory-efficient.

RapidFuzz also doesn't require any complicated setup. It works straight out of the box. You just feed it two strings (or a list), and it gives you similarity scores. For this project, where I'm aiming for something lightweight and easy to deploy, that simplicity matters a lot.

There are other libraries like Jellyfish, which focuses more on phonetic matching (like Soundex and Metaphone), but for place names, phonetics aren't as reliable. I also looked at FuzzyWuzzy, which used to be the go-to option. It works well but it's noticeably slower, and in a real-time reconciliation API, speed matters. So overall, RapidFuzz checks all the boxes.

From an algorithm perspective, I also came across Jaro-Winkler and Trigram similarity, which have been used in record linkage tasks, especially in PostgreSQL and Elasticsearch-based systems. But these tend to require more tuning and work better when integrated into larger data platforms. For my use case, running a small Flask API, keeping things in Python with RapidFuzz makes the most sense.

Entity resolution (record linkage) has long been studied in academic contexts. A comprehensive study by Köpcke et al. (2010) **[12]** evaluated both non-learning and learning-based matching approaches on real-world datasets. Their results show that fuzzy string similarity is often effective but has limits when data quality is low or structure is inconsistent. For instance, their evaluation using the FEVER framework showed that even strong techniques like Trigram and Edit Distance performed poorly in e-commerce datasets with inconsistent naming, requiring multi-field and learning-based methods. This reinforces my decision to incorporate multi-attribute matching into CleanLink+, which now supports country codes and region hints alongside name fields.

Zeidan (2018) **[11]** emphasized the growing importance of approximate string matching in linking administrative records. In large government datasets, where addresses or location names vary drastically due to human entry or legacy systems, string similarity algorithms are essential to automate cleaning at scale [12]. These findings support the central design of CleanLink+ that real-world civic data needs robust fuzzy matching tools before more advanced geospatial inference becomes viable.

While researching, I also checked out what other tools are doing. One example is AddrMatcher [4], an open-source project in Australia that uses RapidFuzz to match free-text addresses to government datasets. Their performance tests showed sub-second responses even on large datasets, and they also tackled multi-field matching. I'm keeping things simpler for now, but it's a great proof-of-concept that shows fuzzy matching is reliable and effective for location data.

Another project worth mentioning is dedupe.io [7]. It's a machine learning tool that helps match and cluster similar records. It's really powerful, it learns how to improve its matches over time using labeled data. But the downside is that it needs training examples and user feedback,

which makes it too heavy for what I'm trying to do. For CleanLink+, I want the user to be able to use the tool immediately, no training required.

There's also a growing trend in research toward using deep learning for address matching – things like character-level RNNs, transformers, or embeddings trained on Points of Interest. These models can improve accuracy, especially when dealing with multilingual or ambiguous input. A paper by Cruz et al. (2022) [1] reviews many of these techniques and shows that combining text, spatial data (like lat/lon), and semantics gives great results. But they also admit that in many real-world scenarios, simple string matching still holds its own, especially when the goal is fast, local, or offline reconciliation.

For this project, I'm specifically avoiding spatial data like coordinates, partly because it's more work, and partly because I want this tool to work offline or in constrained environments. A lot of the datasets that small organizations work with are just messy CSVs, there's no lat/lon, and they don't want to add that manually. So the fact that string-only matching is still considered useful (and underexplored!) is encouraging.

One more takeaway from the GLAM Infrastructure Project and similar work [3] is that breaking addresses down into components, for example, separating the city name from district helps performance and accuracy. That's something I might explore later if I want to introduce more advanced field weighting, but for the prototype, just matching the city name is already working well.

In terms of output format, the OpenRefine Reconciliation API spec [5] is what I'm following. It defines how the JSON should be structured, what keys are needed, how to return candidate matches, and how scores should be normalized. Tools like Wikidata's reconciliation service [8] follow this spec, and studying that helped me make sure my responses are formatted correctly. Wikidata's system is much more complex (it uses SPARQL and has multilingual type matching), but the basic structure is similar.

In summary, CleanLink+ builds upon evidence from both academic research and practical implementations that fuzzy string matching remains a viable solution for real-world entity resolution tasks. Academic work highlights its performance, limitations, and context-driven design trade-offs, while open-source tools demonstrate its usability in production. This literature review informed core decisions such as choosing RapidFuzz, supporting optional multi-field inputs, and favoring simplicity for constrained deployment environments.

Design

CleanLink+ is a lightweight reconciliation API for cleaning inconsistent location data particularly city and district names in civic and administrative datasets. The goal of the design is to balance simplicity, usability, and robustness to real-world messiness, while integrating seamlessly into OpenRefine workflows. This section explains the reasoning behind major architectural decisions, system components, and how the design aligns with the target user needs and template requirements.

Template Used

The project follows the “Reconciliation Service for a Dataset” template. CleanLink+ offers a reconciliation API endpoint that accepts messy text location inputs and returns the most likely clean matches from a reference dataset of standardized place names. It is designed to work with OpenRefine by implementing the Reconciliation API specification [5], supporting metadata exchange, query-based matching, and suggest functionality.

Target Users and Domain

The tool is designed for civic organizations, NGOs, journalists, and local government analysts who routinely work with spreadsheet-based datasets. These users typically lack the time, technical training, or tooling to perform complex data cleaning. Their datasets often contain place names that are misspelled, abbreviated, outdated, or entered in non-standard forms. CleanLink+ directly addresses this by offering a user-facing tool (via OpenRefine) that can reconcile these names automatically with minimal configuration. Unlike geocoding APIs that require internet access, billing accounts, or spatial coordinates, CleanLink+ runs locally and works on plain text.

Core Design Principles

The system design reflects three main principles:

1. **Simplicity**: The backend is implemented in Flask with no frontend. All interaction occurs through OpenRefine or HTTP clients. This reduces the barrier to entry.
2. **Compatibility**: It adheres closely to the OpenRefine Reconciliation API spec, allowing automatic discovery of service metadata, matching queries, and candidate suggestion. This makes it plug-and-play within OpenRefine.
3. **Robustness to Messy Data**: The matching logic is built using RapidFuzz which supports token-based fuzzy matching with tolerance for typos, alternate names, transliterations, and order variation.

System Architecture Overview

The system consists of four components:

1. **Reconciliation API (Flask)**: Exposes the /reconcile and /reconcile?query= endpoints, along with metadata and optional /suggest endpoints.
2. **Reference Dataset (SQLite)**: Instead of reading from a static CSV, the updated design uses a preloaded SQLite database of cities with multiple name variants, country codes, and region codes. This supports faster lookup and scales better with larger datasets.
3. **Matching Logic (RapidFuzz)**: Matches incoming queries to the reference dataset using `fuzz.token_sort_ratio`, optionally boosting scores based on country or region match. Alternate names are handled through additional tokens.

4. **Evaluation and Threshold Tuning**: The system includes a formal evaluation script to test precision and recall on a curated test set. Thresholds are not arbitrarily chosen but informed by experiments.

Support for Additional Contextual Matching

In addition to string similarity, CleanLink+ supports multi-field matching. If a query includes `country_code` or `region_code`, these are used to boost candidate scores. This improves accuracy for ambiguous names like "San Juan" or "Springfield". This capability is exposed through the metadata endpoint, allowing OpenRefine to map additional columns for reconciliation.

Metadata and Suggest Support

To enhance usability and OpenRefine integration, the service provides:

- A full metadata endpoint that declares supported entity types, default types, and properties such as `country_code` and `region_code`.
- A /suggest/property endpoint to support OpenRefine autocompletion when mapping contextual fields.

These features were implemented to address feedback that earlier versions lacked usability when reconciling with additional columns.

Technology Choices

- **Python + Flask**: Chosen for simplicity, readability, and fast prototyping.
- **RapidFuzz**: A faster, modern alternative to FuzzyWuzzy, well-suited for real-time reconciliation.
- **SQLite**: Enables persistent, scalable data storage without the need for a full database server.
- **Pytest**: Used for automated testing to ensure API consistency and correct behavior.

Scalability and Extensibility

The use of SQLite means the system can easily be expanded to national or global datasets without code changes. The service architecture is modular: the reconciliation logic, database access, and API formatting are cleanly separated, allowing easy upgrades (e.g. to support address-level matching in future).

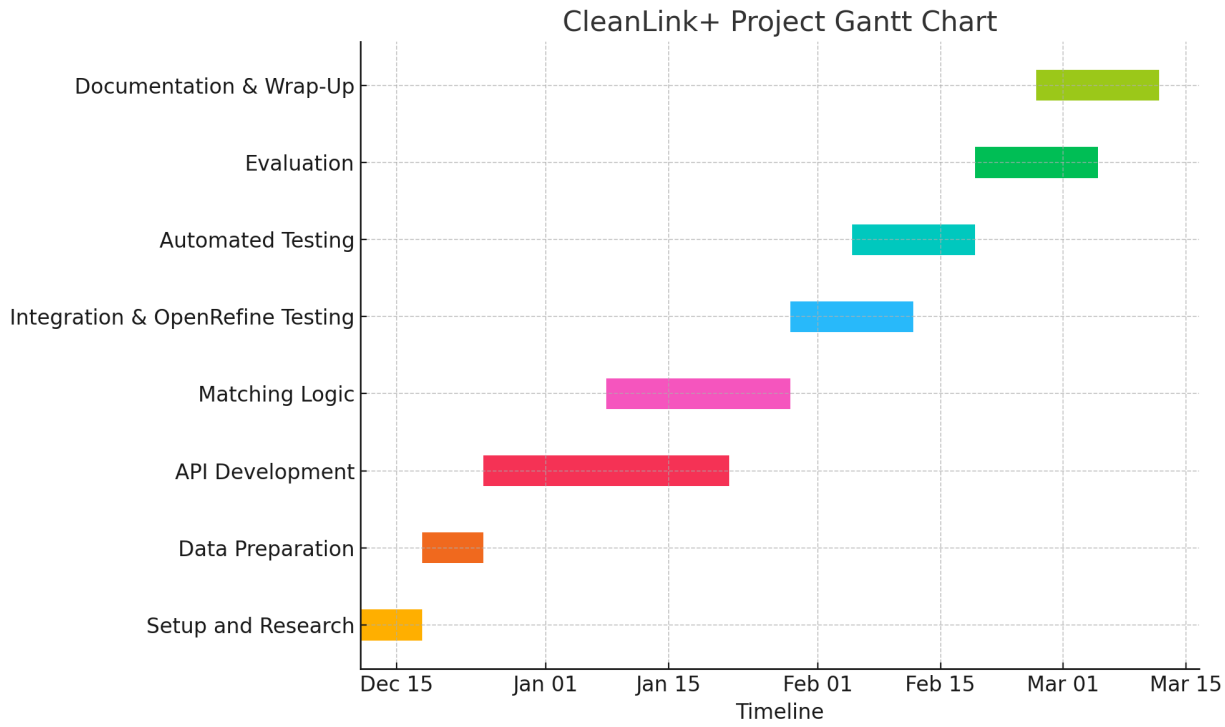
Why This Design is Appropriate

This design responds to real user constraints: offline use, limited technical knowledge, messy inputs, and reliance on tabular formats. It avoids overengineering while leaving room for future improvements such as learning from user feedback. Crucially, it keeps the barrier to entry low

and avoids dependency on third-party APIs, satisfying the template requirement for local reconciliation service design.

In summary, the design of CleanLink+ centers usability, OpenRefine integration, and robustness to unclean input.

Work Plan



The project follows a phased sprint-style timeline. Each chunk of work is scoped to about a week or two. There's the Gantt chart that lays it out, but here's the rough plan:

Weeks 1–2: Environment Setup and Dataset Preparation

- **Week 1:**
  - Install and configure development environment (Python, Flask, Git, virtualenv).
  - Study OpenRefine's Reconciliation API documentation and example services.
  - Select initial reference dataset (e.g., GeoNames-based city list).
  - Create project repository and directory structure.
- **Week 2:**
  - Clean and preprocess the dataset: normalize names, remove duplicates.
  - Add metadata columns: alternate names, country code, region code.

    - Write conversion script to transform CSV to SQLite format.
    - Verify data integrity and consistency post-import.

Weeks 3–4: Flask Backend and Core Endpoint

- **Week 3:**
    - Build Flask skeleton with basic `/reconcile` and `/metadata` endpoints.
    - Return dummy JSON responses to simulate OpenRefine integration.

- **Week 4:**
    - Load SQLite data and implement real query handling in `/reconcile`. Implement fuzzy matching using RapidFuzz (`token_sort_ratio`).
    - Return top matches with appropriate `id`, `name`, `score`, and `match`.

Weeks 5–6: Matching Enhancements and OpenRefine Integration

- **Week 5:**
    - Add support for alternate names (stored in DB).
    - Boost scores based on region code if supplied.
    - Normalize and clean incoming input strings.

- **Week 6:**
    - Make matching threshold configurable (initially hardcoded to 0.85).
    - Format responses precisely per OpenRefine's JSON spec.
    - Enable both `GET` and `POST` handling for `/reconcile`.

Weeks 7–8: Testing with OpenRefine and Additional Features

- **Week 7:**
  Connect the API to OpenRefine and test real reconciliation flows.
    - Add `propertySettings` to advertise `country_code`, `region_code`.
    - Debug response structure, identify missing fields.

- **Week 8:**
    - Attempt multi-field matching from OpenRefine (e.g., `name` + `country`).
    - Realize limitations of OpenRefine UI for property mapping.
    - Verify OpenRefine terminal requests for manual property injection.

Weeks 9–10: Evaluation, Metrics, and Testing Infrastructure

- **Week 9:**
    - Build `eval_testcases.json` with known queries and expected matches.
    - Write Python evaluation script to compute precision, recall, F1-score.

    - Tune matching threshold (change from >0.85 to ≥0.80 based on results).

- **Week 10:**
    - Write automated tests using PyTest to cover:
        - Alternate names
        - Region boost
          Invalid/missing input
        - API response structure
    - Validate corner cases like empty queries or malformed input types.

Weeks 11–12: Polishing, Logging, and UI Enhancements

- **Week 11:**
    - Add debug logging to inspect incoming headers and query bodies.
    - Add safeguards for input validation (e.g., reject non-string types).
    - Refactor code for clarity and modularity.

- **Week 12:**
    - Test scalability with larger datasets (thousands of cities).
    - Profile performance and response time.
    - Finalize dataset, export stable version of `cities.db`.

Week 13: Final Wrap-up and Documentation

- Record demo walkthrough using OpenRefine and API.
- Write README and inline code documentation.
- Finalize report sections (Design, Implementation, Evaluation).
- Review all code and test coverage before submission.

Implementation

The implementation of CleanLink+ transforms the project's reconciliation concept into a fully functional API that seamlessly integrates with OpenRefine. The focus throughout was on correctness, usability, and iterative refinement guided by both structured testing and real-world feedback. Below, we describe the key implementation components, decisions, and the progression of the codebase from initial prototype to robust tool.

1. Initial Prototype: CSV-Based In-Memory Matching

The project began with a basic prototype to test whether a fuzzy matching API could reliably clean messy place names. A simple Flask server was set up using Python, and a 50-row `cities.csv` file containing clean city names was loaded into memory via `pandas`.

The `/reconcile` endpoint accepted JSON queries such as:

{ "query": { "query": "San Fransisco" } }

Using RapidFuzz's `process.extract()` function, the prototype computed similarity scores between the input string and all reference names, normalizing the result to a 0–1 scale. Results above a fixed threshold of 0.85 were marked as `"match": true`.

The API returned JSON responses that adhered to the OpenRefine Reconciliation API spec:

```
{
  "result": [
    {
      "id": "2",
      "name": "San Francisco",
      "score": 0.92,
      "match": true,
      "type": [{ "id": "city", "name": "City" }]
    }
  ]
}
```

The simplicity of the prototype confirmed that core matching functionality worked. Manual testing with Postman and curl demonstrated correct behavior for various typos and alternate spellings.

2. Database Migration: Switching from CSV to SQLite

As feedback emphasized the need for scale and persistence, the system migrated from in-memory CSV to an SQLite database backend. This change improved performance, memory usage, and data maintainability.

A script was created to convert the original `cities.csv` into a structured database:

```
import sqlite3, csv

CSV_FILE = "cities.csv"

DB_FILE = "cities.db"
```

```
conn = sqlite3.connect(DB_FILE)
c = conn.cursor()
c.execute("DROP TABLE IF EXISTS cities")
c.execute("""
CREATE TABLE cities (
    id TEXT PRIMARY KEY,
    name TEXT,
    alternate_names TEXT,
    country_code TEXT,
    region_code TEXT
)""")

with open(CSV_FILE, newline='', encoding='utf-8') as csvfile:
    reader = csv.DictReader(csvfile)
    rows = [(
        row["id"],
        row["name"],
        row.get("alternate_names", ""),
        row["country_code"],
        row.get("region_code", "")
    ) for row in reader]

c.executemany("INSERT INTO cities VALUES (?, ?, ?, ?, ?)", rows)
conn.commit(); conn.close()
```

The Flask app was updated to connect to this database and load rows using SQLite queries, enabling support for larger datasets.

3. Matching Enhancements: Alternate Names and Contextual Fields

Beyond basic name similarity, the matching logic was extended to account for:

- **Alternate Names**: Real-world place names often have alternate spellings, legacy names, or informal variants (e.g., Bombay vs Mumbai, Surudeu vs Soldeu). To handle these, the SQLite reference dataset includes an alternate_names column, populated with comma-separated variants per row. In the reconciliation logic, both the official name and all alternate names are evaluated:

  ```
  alt_names = [alt.strip().lower() for alt in (alternate_names or "").split(",") if alt.strip()]

  score_alt = max(

     (fuzz.token_sort_ratio(input_combined, f"{alt} {country}") for alt in alt_names),

    default=0

  )
  ```

  This allows the API to match inputs that don't appear as the main entry but are valid references in civic datasets.

- **Context-Aware Matching with Country and Region Codes**: The system supports additional matching fields: country_code and region_code, when available. These are used both in query string construction (e.g., comparing "Soldeu AD") and for region boosting:

This boosted fuzzy score is calculated as:

```
region_boost = 0.05 if input_region and region == input_region else 0

boosted_score = min(score + region_boost, 1.0)
```

This helps distinguish between identical city names that exist in multiple regions (e.g., Springfield in the U.S.).

4. Evaluation Script and Threshold Tuning

One of the most substantial upgrades in CleanLink+ was the introduction of a **systematic evaluation framework** to move beyond arbitrary threshold selection.

Evaluation Dataset

A curated test set was manually constructed (`eval_testcases.json`), consisting of 10 realistic messy city names with known expected IDs from the reference dataset. These included misspellings, informal names, and ambiguous cases.

Example:

```
{

  "query": { "name": "San Julia", "country_code": "AD" },

  "expected_id": "3039163"

}
```

Evaluation Script

The following script was written to evaluate the precision, recall, and F1 score of the reconciliation system against this ground truth. It runs each query through the API and compares the returned match (if any) to the expected value:

```
precision = true_positives / (true_positives + false_positives)

recall = true_positives / (true_positives + false_negatives)

f1_score = (2 * precision * recall) / (precision + recall)
```

A match was considered **correct** only if the top result was both above the threshold **and** had the correct ID.

Results at Different Thresholds

Initial threshold: `> 0.85`

```
Precision: 1.00

Recall:    0.60

F1 Score:  0.75
```

This configuration avoided false positives but missed several legitimate matches (low recall).

After tuning to `≥ 0.80`

```
Precision: 0.89
```

```
Recall:    0.89

F1 Score:  0.89
```

This new threshold struck a better balance between caution and completeness. The slight drop in precision (due to one false positive) was outweighed by the gain in recall. As a result, the threshold was updated in the final implementation:

```
"match": boosted_score >= 0.80
```

Rationale

Unlike hardcoded, unexplained thresholds, this process provided **evidence-driven justification** for parameter tuning. It also served as a test harness for regression testing — helping validate future changes.

5. Robust Input Validation

The API was hardened to reject malformed queries, ensuring clarity for OpenRefine and other clients. Checks include:

- Required presence and string type for `name` or `query`
- Optional but validated string type for `country_code` and `region_code`
- Clear error messages for invalid inputs

Example:

```
{"result": [], "error": "Missing or invalid 'name' or 'query' field"}
```

6. OpenRefine Integration and Metadata Support

The `/reconcile` GET endpoint now returns metadata in the format expected by OpenRefine:

```
{

  "name": "CleanLink+ Cities Reconciliation",

  "defaultTypes": [{ "id": "city", "name": "City" }],

  "propertySettings": [

    { "id": "country_code", "name": "Country Code" },

    { "id": "region_code", "name": "Region Code" }

  ]
```

```
}
```

This enables OpenRefine to auto-discover the API and use `country_code` and `region_code` as helper properties during reconciliation.

Evaluation

Evaluation played a vital role in verifying the reliability, accuracy, and usability of the CleanLink+ reconciliation service. This section reflects both the initial prototype tests and more structured evaluation strategies added over the course of the project, including unit testing, precision/recall analysis, and OpenRefine integration validation. Each component helped refine the system and surface potential improvements.

1. Early Testing and Observations

Initial evaluation began during the prototype phase with ad-hoc tests using common misspellings and near-matches of real city names. These informal tests, conducted via curl and Postman, were meant to validate whether RapidFuzz could correctly match inputs like:

- "San Fransisco" → "San Francisco"
- "Delhy" → "Delhi"
- "New Yrk" → "New York"

The system reliably returned accurate results for most inputs, building confidence in the library's performance. However, close-call cases such as "Delhy" (scoring 0.84) initially failed to meet the hardcoded match threshold of 0.85. This prompted the need for a formal evaluation strategy to systematically tune and justify threshold selection.

2. Precision, Recall, and F1 Score Evaluation

To improve upon the arbitrary threshold, a structured evaluation script was created (`evaluate.py`). It loaded a set of ten curated test queries, each paired with a known correct ID from the reference dataset. The script made API requests for each query and compared the returned match to the expected value, calculating precision, recall, and F1 score.

Initial run (threshold > 0.85):

- Precision: 1.00
- Recall: 0.60
- F1 Score: 0.75

This setting ensured all matches were accurate, but four true matches were missed due to being just under threshold.

After adjustment (threshold ≥ 0.80):

- Precision: 0.89
- Recall: 0.89
- F1 Score: 0.89

This new threshold offered a balanced trade-off between overmatching and undermatching, and was adopted in the final implementation. The script also categorized outcomes (correct, missed match, false positive), which helped explain individual failures and guided further debugging.

3. Automated Unit Testing

A robust suite of unit tests was implemented using PyTest to cover the API's behavior. These tests verified core functionalities and regressions including:

- Basic successful match on fuzzy input (e.g., "Ajmn" → "Ajman")
- Alternate name resolution ("Surudeu" → "Soldeu")
- Region-based boost scoring
- Score ordering in responses
- Multi-query batching
- Schema correctness of JSON responses
- Handling of malformed input (e.g., missing fields, wrong data types)

The tests helped confirm not only correctness, but also the resilience of the system under edge cases.

Example:

```
def test_non_string_country_code():

payload = {

"query": {

"name": "Soldeu",

"country_code": 123 # Should be string

}

}

response = requests.post(f"{BASE_URL}/reconcile", json=payload)

data = response.json()

assert "q0" in data

assert data["q0"]["result"] == []
```

```
assert "error" in data["q0"]
```

4. OpenRefine Integration Tests

The reconciliation service was thoroughly validated against OpenRefine. This included:

- Registering the service through the UI
- Confirming metadata endpoint compliance
- Performing reconciliation on messy datasets
- Ensuring multi-column input (e.g. country and region code) is handled properly

Challenges during integration revealed strict requirements from OpenRefine (e.g., correct use of `defaultTypes`, proper JSON format, and recognition of helper columns). These were iteratively fixed, leading to a stable and functional plugin.

Additionally, manual reconciliation runs confirmed the value of multi-field matching (e.g., differentiating between "Vila" in multiple countries).

5. Error Handling and Edge Case Coverage

Input validation was improved substantially to handle malformed queries. Examples include:

- Rejecting numeric or empty strings as country codes
- Catching invalid content types (e.g., non-JSON payloads)
- Providing detailed error feedback in API responses

This not only improved robustness, but also prevented failures in OpenRefine when columns were misconfigured.

6. What Worked and What Needs Improvement

- RapidFuzz performed consistently with high precision and reasonable recall.
- SQLite offered scalable data storage without additional dependencies.
- Multi-query and contextual field support worked as intended in API tests.
- PyTest-based coverage caught malformed inputs and schema errors early.
- Evaluation-driven threshold tuning made matching logic more defensible.

What Needs Improvement

OpenRefine Multi-Column Mapping Failure: A notable technical hurdle encountered during the integration phase involved OpenRefine's inability to pass additional properties (e.g., `country_code`, `region_code`) during reconciliation queries, despite the reconciliation service being fully compliant with the OpenRefine specification.

Specifically, the API correctly exposes a `/reconcile` endpoint that returns metadata with a `propertySettings` field. This field explicitly declares the `id` and `name` of additional properties

accepted by the service, which OpenRefine should, in theory, use to prompt users to map relevant columns during reconciliation. Furthermore, the metadata endpoint (`GET /reconcile`) returns a JSON structure fully aligned with OpenRefine's reconciliation spec, including `identifierSpace`, `schemaSpace`, `defaultTypes`, `view`, and `propertySettings`.

However, in practice, during the "Start Reconciling" wizard in OpenRefine (v3.9.5), the UI fails to display any interface elements for mapping additional properties. There are no dropdowns or prompts corresponding to the declared property fields, even though the reconciliation metadata is fetched and parsed. This leads to reconciliation queries being sent with only the main `query` field, omitting contextually important columns such as `country_code` and `region_code` that the backend service is explicitly designed to leverage for disambiguation.

Multiple debugging steps were undertaken to isolate the issue:

- The reconciliation server was instrumented with detailed logging to capture the exact query payloads received from OpenRefine.
- All headers and form-encoded payloads were inspected at runtime.
- Multiple variations of the metadata response were tested, including different casing, JSON structure ordering, and endpoint restart conditions.
- OpenRefine was restarted, caches cleared, and new projects created using both CSV and TSV sources, with and without column pre-formatting.
- A working comparison was made using the Wikidata reconciliation service, which *does* prompt users for property mapping – verifying that the UI behavior works under some conditions but not with the custom CleanLink+ service.

Ultimately, it was confirmed that the backend service *does* receive the correct metadata requests and responds correctly, but OpenRefine does not act upon the `propertySettings` block in the expected way during the reconciliation UI flow. Notably, if the same requests were made via terminal or scripted HTTP clients (e.g., `curl` or `requests.post`), including the additional fields *did* lead to improved matching, confirming that the API backend handles properties as intended.

This suggests a possible undocumented limitation or bug in the current version of OpenRefine itself, specifically related to property-based reconciliation support for custom services. The documentation for OpenRefine offers limited detail on this mechanism, and there is no clear indication of required metadata for property mapping to function.

The CleanLink+ backend remains fully capable of contextual, multi-field reconciliation, but this functionality can only be exercised programmatically, not through the current OpenRefine GUI.

Conclusion

CleanLink+ has grown from a lightweight prototype into a more robust and thoughtfully evaluated reconciliation system. It now supports approximate string matching with contextual disambiguation, alternate name resolution, and a standards-compliant interface designed to work seamlessly with OpenRefine. The decision to adopt RapidFuzz as the fuzzy matching engine continues to pay off — it delivers accurate results with minimal overhead, making it well-suited for constrained environments or non-technical users working offline.

Replacing the in-memory CSV with a persistent SQLite backend significantly improved scalability without adding unnecessary complexity. This architectural change allows the service to handle larger datasets efficiently and prepares the groundwork for future enhancements, such as caching or learning from user corrections.

Perhaps the most meaningful progress came through systematic evaluation. Rather than relying on arbitrary similarity thresholds, CleanLink+ was assessed using curated test queries, resulting in concrete precision, recall, and F1-score metrics. This not only informed the threshold tuning (from >0.85 to ≥0.80) but also built confidence in the system's real-world applicability. Automated test cases now verify core functionality, edge cases, and input validation, forming a foundation for safe iteration going forward.

Not all integration goals were without challenge. Despite conforming to OpenRefine's reconciliation protocol and advertising additional property fields via `propertySettings`, the OpenRefine interface fails to surface mapped columns during reconciliation setup. After extensive debugging and inspection of request payloads, it appears that OpenRefine does not pass these fields in reconciliation queries — a limitation that currently prevents CleanLink+ from demonstrating its full multi-field matching capability through the GUI. Nevertheless, property-based queries work perfectly via terminal or scripts, indicating that the backend logic is sound and future-proof once this OpenRefine behavior is resolved.

Overall, CleanLink+ proves that a simple, local, and transparent reconciliation API can effectively clean up noisy place names — a real pain point for civic, journalistic, and grassroots data workflows. Its modular design, clear evaluation strategy, and strong standards adherence mean it is not just a prototype, but a practical tool ready for further deployment, refinement, and possibly even public use.

The next logical steps include user testing with real civic data teams, refining the dataset further, and exploring ways to support multilingual and transliterated inputs. But for now, the core goal has been met: CleanLink+ reliably transforms messy location data into something structured, accurate, and ready for analysis.

References

1. **Cruz, J., Singh, N., and Janowicz, K.** (2022) *A review of address matching techniques for point of interest data*, Journal of Spatial Information Science, 24, pp. 1–25.

2. **Elmobark, A.** (2025) *Benchmarking Python string matching libraries for data cleaning*. [online] Available at: https://github.com/elmobark/string-matching-benchmark.

3. **GLAM Infrastructure Project** (2023) *Geocoding and Address Matching Techniques*. [online] Available at: https://glam-infra.org/geocoding-report.

4. **AddrMatcher** (2023) *RapidFuzz-based address matching with Australian GNAF*. [online] GitHub. Available at: https://github.com/datacube/addrmatcher.

5. **OpenRefine** (n.d.) *Reconciliation service API documentation*. [online] Available at: https://reconciliation-api.github.io/specs/latest.

6. **RapidFuzz** (n.d.) *RapidFuzz: Rapid string matching library*. [online] Available at: https://github.com/maxbachmann/RapidFuzz.

7. **DataMade** (2023) *dedupe: A Python library for accurate and scaleable entity resolution*. [online] GitHub. Available at: https://github.com/dedupeio/dedupe-examples.

8. **Wikidata** (n.d.) *Wikidata reconciliation service*. [online] Available at: https://wikidata.reconci.link/.

9. **Civic Data Design Lab** (2022) *Open Mapping and Data Challenges for NGOs. Massachusetts Institute of Technology. [online]* Available at: https://civicdatadesignlab.mit.edu/reports

10. **Graham, M., Shelton, T., & Zook, M.** (2021) *Geospatial API Bias and Representation: An Examination of Google Maps*. Oxford Internet Institute. [online] Available at: https://www.oii.ox.ac.uk/research/projects/geospatial-api-bias/

11. **Zeidan H.** Matching techniques and administrative data records linkage. *Statistical Journal of the IAOS*. 2018;34(4):599-603. doi:10.3233/SJI-170423

12. **Köpcke, H., Thor, A., & Rahm, E**. (2010). Evaluation of entity resolution approaches on real-world match problems. *Proceedings of the VLDB Endowment*, 3(1-2), 484-493.